

Sprawozdanie: Porównanie implementacji algorytmu Dijkstry

Marcel Musiałek (Nr Indeksu: 279704)

27 stycznia 2026

1 Opis implementacji i złożoność obliczeniowa

W ramach zadania zaimplementowano trzy warianty algorytmu wyznaczania najkrótszych ścieżek w grafie z nieujemnymi wagami krawędzi.

1.1 Algorytm Dijkstry (Wariant podstawowy)

Implementacja wykorzystuje standardową bibliotekę C++ (STL). Jako kolejkę priorytetową zastosowano kontener `std::priority_queue`, który jest implementacją kopca binarnego.

- **Szczegóły implementacji:** Użyto techniki *lazy deletion* – wierzchołki nie są usuwane z kopca przy aktualizacji odległości, lecz dodawane ponownie z nowym priorytetem. Nieaktualne wpisy są ignorowane przy zdejmowaniu z kolejki.
- **Złożoność obliczeniowa:** Operacje na kopcu binarnym zajmują $O(\log K)$, gdzie K to liczba elementów w kopcu (w najgorszym razie $K \approx m$). Całkowita złożoność wynosi $O(m \log n)$.

1.2 Algorytm Diala

Jest to implementacja algorytmu Dijkstry wykorzystująca strukturę kubełkową (buckets). Każdy kubełek k przechowuje listę wierzchołków o tymczasowej odległości równej k .

- **Szczegóły implementacji:** Ze względu na specyfikę algorytmu, liczba kubełków jest zależna od maksymalnej wagi krawędzi C . Algorytm porusza się cyklicznie lub liniowo po kubełkach.
- **Złożoność obliczeniowa:** $O(m + C)$, gdzie C to maksymalny koszt (waga) krawędzi.
- **Ograniczenia:** Algorytm jest bardzo wydajny dla małych wag całkowitych, ale jego złożoność pseudowielomianowa czyni go nieużywalnym dla grafów o dużych wagach (np. rodziny Long-C), gdzie czas wykonania drastycznie rośnie.

1.3 Algorytm Radix Heap

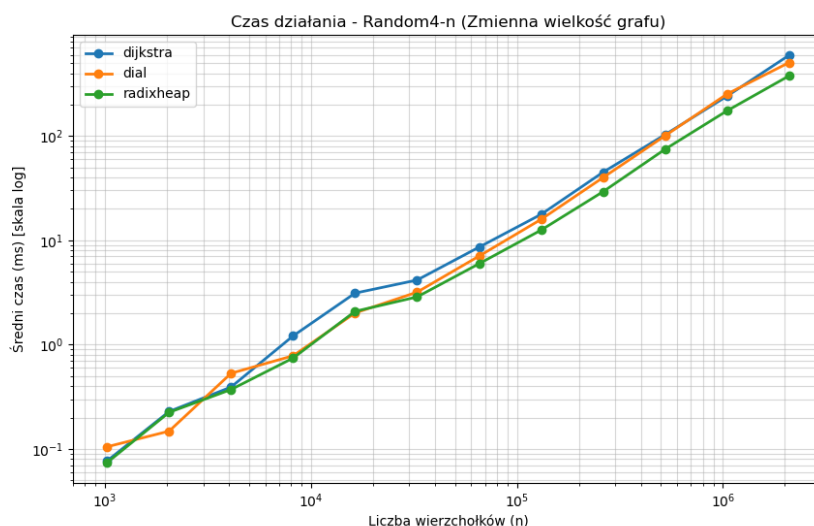
Zoptymalizowana wersja algorytmu kubełkowego, wykorzystująca własności reprezentacji binarnej liczb. Kubełki przechowują wierzchołki w zakresach odległości $[x, x + 2^k)$.

- **Szczegóły implementacji:** Zastosowano funkcję wbudowaną kompilatora GCC `__builtin_clzll` (Count Leading Zeros) do błyskawicznego wyznaczania indeksu kubełka w czasie $O(1)$. Zastosowano mechanizm *lazy deletion* oraz bezpieczną redystrybucję elementów z wykorzystaniem `std::swap` (w celu uniknięcia błędów segmentacji).
- **Złożoność obliczeniowa:** $O(m + n \log C)$.
- **Ograniczenia:** Implementacja jest wrażliwa na przepełnienie zakresów (integer overflow) przy ekstremalnie dużych odległościach skumulowanych (np. w grafach Long-n o dużej średnicy), co może prowadzić do błędów wykonania.

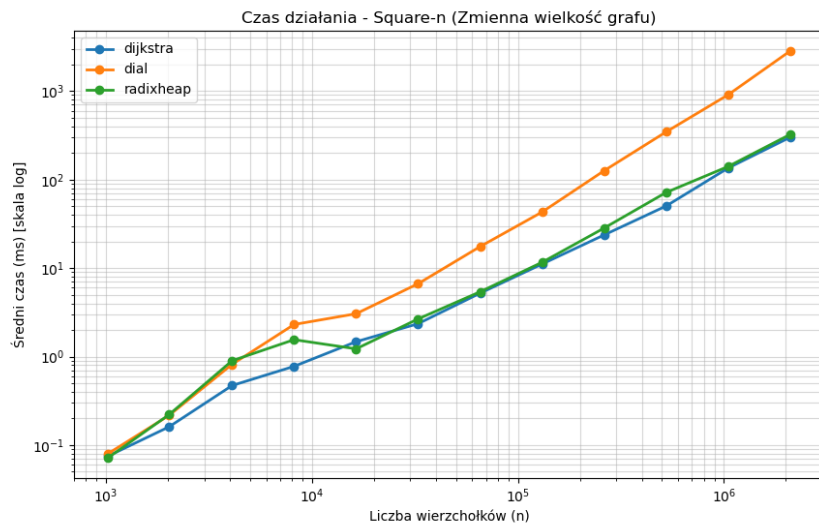
2 Wyniki eksperymentów

Testy przeprowadzono na komputerze z systemem Linux (CachyOS). Dla każdego testu mierzono średni czas wyznaczenia najkrótszych ścieżek od źródła do wszystkich wierzchołków (tryb SS), uśredniony dla 1 ustalonego i 5 losowych źródeł.

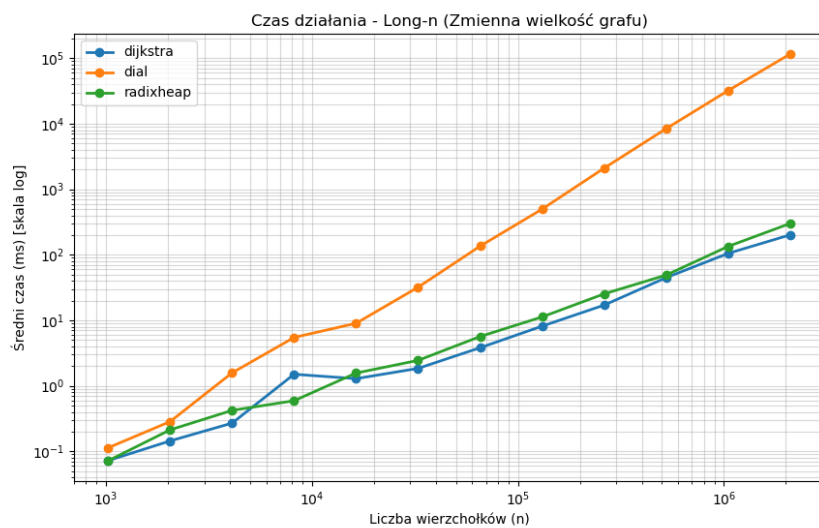
2.1 Wykresy czasu działania



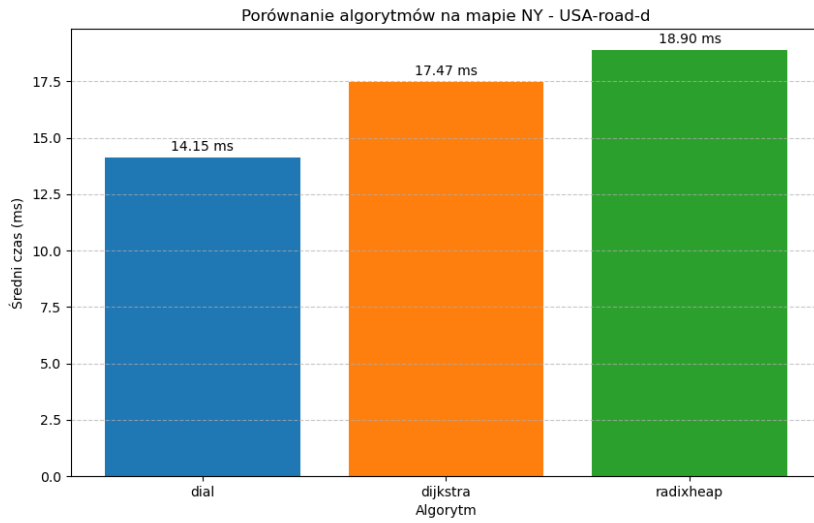
Rysunek 1: Czas działania dla rodziny Random4-n (zmienna n).



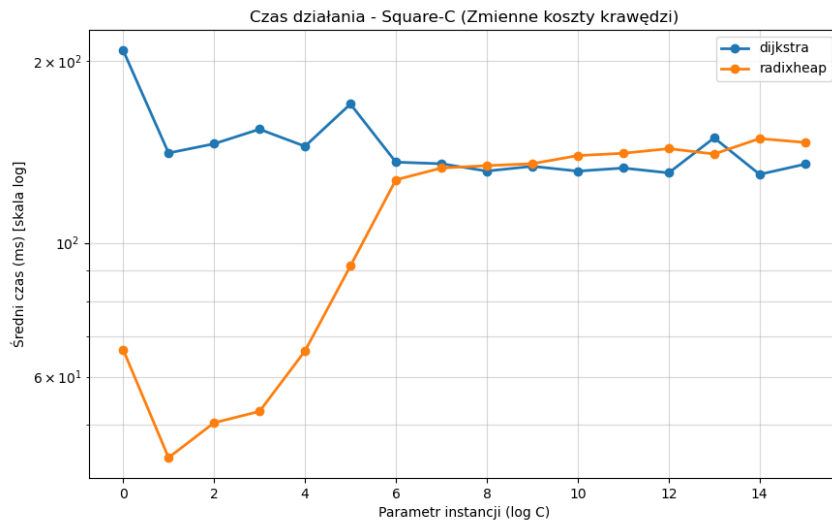
Rysunek 2: Czas działania dla rodziny Square-n (zmiennie n).



Rysunek 3: Czas działania dla rodziny Long-n (zmiennie n).



Rysunek 4: Porównanie algorytmów dla mapy drogowej USA (NY).



Rysunek 5: Czas działania dla rodziny Square-C (zmienne wagi C).

2.2 Wyniki P2P dla największych instancji

Poniższa tabela przedstawia długości najkrótszych ścieżek dla wybranych par wierzchołków w największych badanych grafach.

Tabela 1: Długości ścieżek (P2P) dla największych instancji.

Rodzina	Plik	Para (u, v)	Dystans
Random4-n	Random4-n.21.0.gr	(1, 2097152)	9 051 281
Square-n	Square-n.21.0.gr	(1, 2096704)	714 640 488
Long-n	Long-n.21.0.gr	(1, 2097152)	31 336 751 771
USA-road	USA-road-d.NY.gr	(1, 264346)	495 306

3 Interpretacja wyników i wnioski

Analiza przeprowadzonych eksperymentów pozwala na sformułowanie następujących wniosków:

1. **Stabilność algorytmu Dijkstry:** Standardowa implementacja z kopcem binarnym ($O(m \log n)$) okazała się najbardziej uniwersalna. Działała stabilnie dla wszystkich rodzin grafów, niezależnie od struktury czy wielkości wag. Jest to najbezpieczniejszy wybór do ogólnych zastosowań, co widać na wykresach – linia Dijkstry nigdy nie "wystrzeliła" w górę tak gwałtownie jak konkurencja.
2. **Niejednorodna wydajność algorytmu Diala:**
 - Algorytm "zabłysnął" na rzeczywistej mapie drogowej (USA-road-d), gdzie wyprzedził Dijkstrę (14.15 ms vs 17.47 ms).
 - Jednak w testach syntetycznych (rodziny Square-n i Long-n), mimo małych wag, algorytm okazał się najwolniejszy. Widać, że algorytm tracił czas na iterowanie po pustych kubelkach, co przy takiej strukturze grafu niweluje jego teoretyczną przewagę. Zysk z braku kopca został "zjedzony" przez konieczność sprawdzania pustych indeksów w tablicy.
 - Dla dużych wag (rodziny -C) algorytm zgodnie z teorią stał się nieużywalny.
3. **Specyfika Radix Heap:**
 - Ten algorytm dobrze radził sobie w testach syntetycznych (Square-n), często dorównując lub nieznacznie wyprzedzając Dijkstrę.
 - Co ciekawe, na mapie USA (Rysunek 4) Radix Heap wypadł najgorzej (18.90 ms). Sugeruje to, że narzut na zarządzanie bardziej skomplikowaną strukturą danych (operacje bitowe, przenoszenie kubelków) nie zawsze zwraca się w praktyce, jeśli graf nie jest specyficznie dopasowany pod ten algorytm.
 - Podobnie jak w przypadku Diala, przy bardzo dużych odległościach (Long-n) pojawiły się problemy z przekroczeniem zakresów zmiennych (overflow).
4. **Wpływ topologii:** Struktura grafu ma kluczowe znaczenie. Algorytmy kubelkowe (Dial) zyskały przewagę jedynie na gęstej siatce mapy drogowej. Na rzadszych grafach syntetycznych ich wydajność spadała, przez co standardowy Dijkstra okazał się "złotym środkiem" – może nie zawsze najszybszym, ale najbardziej przewidywalnym.